



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

ASIGNATURA: DESARROLLO DE APLICACIONES MÓVILES

Laboratorio 2: Aplicar Paradigma Atomic Design y componentes UI

Arquitectura e Interfaces en Flutter

El documento presenta la resolución de ejercicios algorítmicos aplicados al desarrollo móvil, incluyendo el manejo de navegabilidad y la implementación visual de un Splash screen. Se estructuran las lógicas de negocio requeridas usando pseudocódigos, omitiendo el uso de diagramas de flujo de datos.

Autores: Diana Guerra, Alejandro Montufar, Paul Moreno, Leonel Tipán

Fecha: 12 de mayo de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Marco Teórico	2
3.1. Flutter	2
3.2. Dart	2
3.3. Android Studio	2
3.4. Splash Screen	3
3.5. Navegabilidad (push, pop, pushNamed)	3
4. Desarrollo de los Ejercicios	3
4.1. Splash Screen y Pantalla Principal	3
4.2. Problema 1: Cálculo de IVA y Descuento	6
4.2.1. Pseudocódigo	6
4.2.2. Diagrama Nassi-Shneiderman	7
4.2.3. Código	7
4.2.4. Ejecución	8
4.2.5. Salida Esperada	9
4.3. Problemas 2	10
4.3.1. Ejercicio 4.1: Incremento salarial	10
4.3.2. Ejecución	11
4.3.3. Salida Esperada	12
4.3.4. Ejercicio 4.2: El naufrago satisfecho	13
4.3.5. Ejecución	15
4.3.6. Salida Esperada	16
4.3.7. Ejercicio 4.3: Contar cantidades	17
4.3.8. Ejecución	18
4.3.9. Salida Esperada	19
4.3.10. Ejercicio 4.8: Promoción de artículos	20
4.3.11. Ejecución	22
4.3.12. Salida Esperada	23
5. Conclusiones	24
6. Referencias	24
7. Anexos	24
7.1. Repositorio de código fuente	24
7.1.1. Enlace principal	24

1. Introducción

El presente documento detalla la resolución técnica de casos prácticos enfocados en el desarrollo de aplicaciones móviles multiplataforma utilizando Flutter y el lenguaje Dart. Estos ejercicios correspondientes al Laboratorio 2, abarcan la aplicación del Paradigma *Atomic Design* para lograr una separación clara y modularidad en la interfaz de usuario, así como la correcta gestión de la navegabilidad. A través del uso de pseudocódigos, se modeló la lógica de negocio antes de su posible implementación visual, garantizando una codificación estructurada y eficiente.

2. Objetivos

- **Objetivo General:** Desarrollar aplicaciones móviles en Flutter aplicando el paradigma de *Atomic Design* y mecánicas de navegación fluidas, estructurando lógicas algorítmicas claras.
- **Objetivo Específico 1:** Implementar y analizar un *Splash screen* (pantalla de bienvenida) inicial que cargue el *branding* al ejecutar la aplicación.
- **Objetivo Específico 2:** Investigar las instrucciones de navegabilidad `push`, `pop` y `pushNamed` para construir flujos lógicos en la pila de *Widgets*.
- **Objetivo Específico 3:** Modelar las soluciones lógicas de problemas matemáticos y de facturación a través de pseudocódigos de forma estructurada, prescindiendo del modelado mediante DFD.

3. Marco Teórico

3.1. Flutter

Flutter es un framework de desarrollo multiplataforma creado por Google que permite construir interfaces nativas para Android, iOS, web y escritorio desde una sola base de código. Su paradigma principal es la programación declarativa con *widgets*, donde la interfaz se describe como un árbol de componentes reutilizables. Flutter utiliza un motor gráfico propio para renderizado, lo que reduce la dependencia de componentes nativos y mantiene consistencia visual entre plataformas.

3.2. Dart

Dart es el lenguaje de programación usado por Flutter. Combina tipado estático opcional, soporte para programación orientada a objetos y características modernas como `async/await` para la concurrencia. Su compilador permite generar código nativo de alto rendimiento para dispositivos móviles y código JavaScript para la web, lo que habilita el enfoque multiplataforma de Flutter.

3.3. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para Android y uno de los más usados para Flutter. Proporciona herramientas de edición, depuración, emuladores y

administración de dependencias. En proyectos Flutter integra la ejecución en dispositivos, el análisis estático y el soporte para hot reload, lo que acelera el ciclo de desarrollo.

3.4. Splash Screen

Un *Splash screen* (o pantalla de bienvenida) es la primera imagen o animación que aparece al abrir una app móvil, mostrando el logo y branding mientras se carga el contenido inicial. Es un recurso clave en la interfaz gráfica para notificar al usuario que la app está en proceso de inicialización, ocultando posibles tiempos de espera ocasionados por consultas asíncronas iniciales.

3.5. Navegabilidad (push, pop, pushNamed)

La navegación en Flutter se sustenta en el widget `Navigator`, el cual maneja una estructura de tipo pila (*Stack*) de pantallas (rutas). Sus comportamientos principales son:

- **push:** Añade de manera imperativa una nueva ruta (*Widget*) en la cima de la pila, transicionando la vista del usuario a esa nueva pantalla.
- **pop:** Remueve la ruta en la cima de la pila, haciendo que la pantalla actual retroceda o vuelva a la interfaz anterior (actúa como el clásico botón “*^*trás”).
- **pushNamed:** Incorpora una ruta a la pila haciendo uso de su nombre (*String*) el cual se declara de antemano en el mapa de rutas (`routes`) del `MaterialApp`. Esto mejora la escalabilidad del código.

4. Desarrollo de los Ejercicios

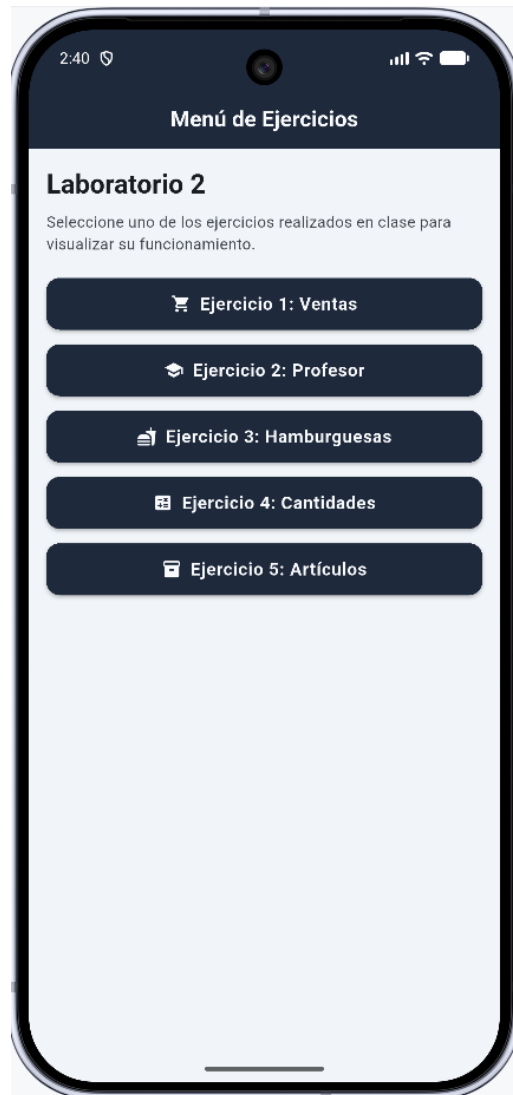
4.1. Splash Screen y Pantalla Principal

Descripción:

Uso de un splash screen (pantalla de bienvenida) que consiste en la primera imagen o animación al abrir la app móvil, mostrando el logo.



Ejecución del Splash Screen



Ejecución de la Pantalla Principal

4.2. Problema 1: Cálculo de IVA y Descuento

Descripción:

Indicaciones al ejercicio en clase: calcular el iva de 15 %, si las compras superan 2000\$ al cliente se agrega un descuento del 20 %, mostrar tanto el sueldo a ganar del vendedor y la factura de la venta de productos.

4.2.1. Pseudocódigo

Inicio

```
Leer subtotalCompra
Leer sueldoBaseVendedor
Leer porcentajeComisionVenta
```

```
iva = subtotalCompra * 0.15
totalConIva = subtotalCompra + iva
```

```
descuento = 0
Si subtotalCompra > 2000 Entonces
    descuento = totalConIva * 0.20
FinSi
```

```
totalFactura = totalConIva - descuento
```

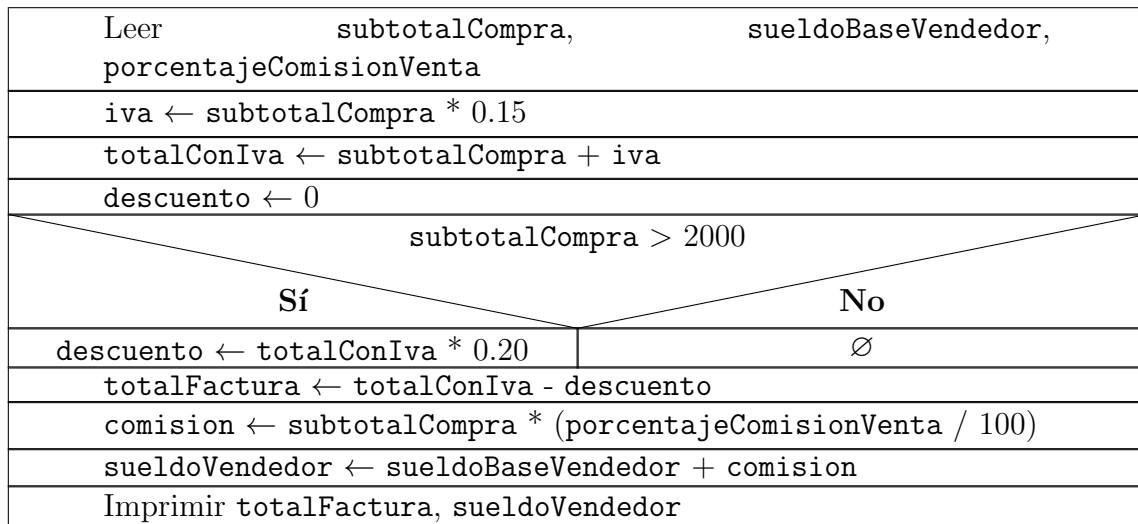
```
comision = subtotalCompra * (porcentajeComisionVenta / 100)
sueldoVendedor = sueldoBaseVendedor + comision
```

```
Imprimir "--- Factura del Cliente ---"
Imprimir "Subtotal: $", subtotalCompra
Imprimir "IVA (15%): $", iva
Imprimir "Descuento: $", descuento
Imprimir "Total a Pagar: $", totalFactura
```

```
Imprimir "--- Sueldo Vendedor ---"
Imprimir "Sueldo a ganar: $", sueldoVendedor
```

Fin

4.2.2. Diagrama Nassi-Shneiderman



4.2.3. Código

•
•
•
lib/model/factura_model.dart

```

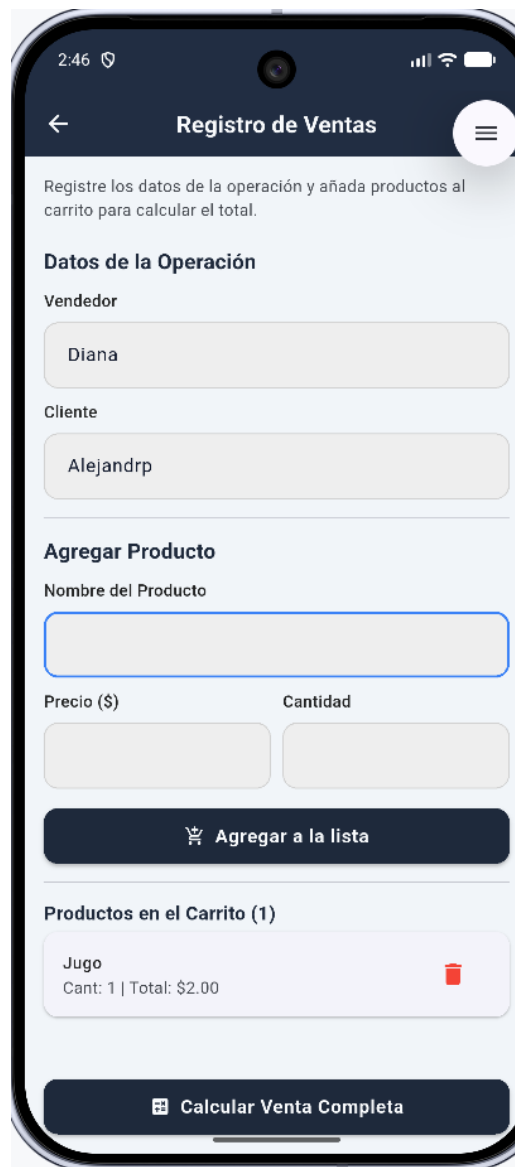
void procesarFactura(double subtotal, double sueldoBase, double comisionPorcentaje) {
  double iva = subtotal * 0.15;
  double totalConIva = subtotal + iva;

  double descuento = 0.0;
  if (subtotal > 2000) {
    descuento = totalConIva * 0.20;
  }
  double totalFactura = totalConIva - descuento;

  double comision = subtotal * (comisionPorcentaje / 100);
  double sueldoVendedor = sueldoBase + comision;
  // Retornar o actualizar estado con los resultados
}
```

Descripción: La función recibe el subtotal de la compra y los datos salariales del vendedor. Calcula el IVA, evalúa si aplica el descuento por compras mayores a 2000, y finalmente determina el salario del vendedor basándose en su sueldo base y comisión.

4.2.4. Ejecución



2:46

← Registro de Ventas

Registre los datos de la operación y añada productos al carrito para calcular el total.

Datos de la Operación

Vendedor

Diana

Cliente

Alejandr

Agregar Producto

Nombre del Producto

Precio (\$) Cantidad

Agregar a la lista

Productos en el Carrito (1)

Jugo
Cant: 1 | Total: \$2.00

Calcular Venta Completa

Ejecución del Problema 1

4.2.5. Salida Esperada



Salida del Problema 1

4.3. Problemas 2

4.3.1. Ejercicio 4.1: Incremento salarial

Descripción:

Un profesor tiene un salario inicial de \$1500, y recibe un incremento de 10 % anual durante 6 años. ¿Cuál es su salario al cabo de 6 años? ¿Qué salario ha recibido en cada uno de los 6 años? Realice el algoritmo utilizando el ciclo apropiado.

Pseudocódigo:

Inicio

 salario = 1500

 Para año = 1 Hasta 6 Hacer

 incremento = salario * 0.10

 salario = salario + incremento

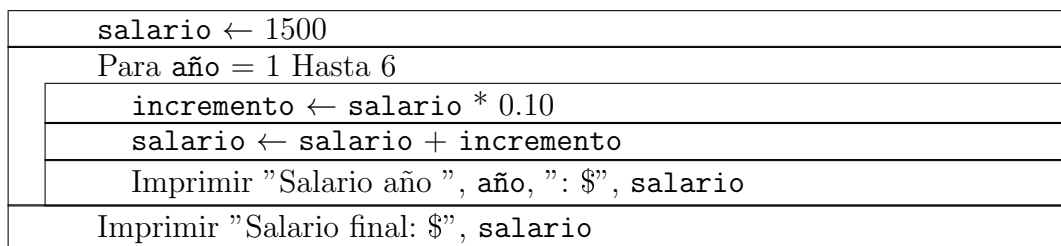
 Imprimir "Salario recibido en el año ", año, ": \$", salario

 FinPara

 Imprimir "El salario final al cabo de los 6 años es: \$", salario

Fin

Diagrama Nassi-Shneiderman:



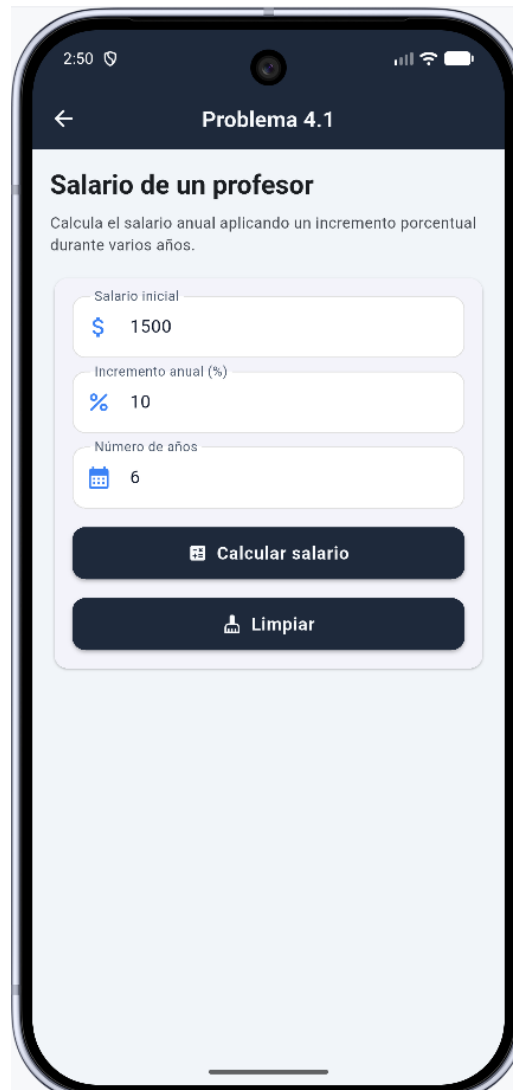
Código:

```
lib/model/profesor_model.dart

void calcularIncrementos() {
  double salario = 1500.0;
  for (int i = 1; i <= 6; i++) {
    double incremento = salario * 0.10;
    salario += incremento;
    print("Salario año $i: $salario");
  }
  // El salario final al cabo de 6 años está en 'salario'
}
```

Descripción: A través de un bucle **for** que itera 6 veces, el algoritmo toma el salario base y calcula de forma acumulativa el 10 % de incremento año tras año.

4.3.2. Ejecución



The screenshot shows a mobile application interface titled "Problema 4.1" with a back arrow. The main heading is "Salario de un profesor". Below it, a subtitle reads: "Calcula el salario anual aplicando un incremento porcentual durante varios años." The interface contains three input fields: "Salario inicial" with a dollar sign icon and the value "1500", "Incremento anual (%)" with a percent sign icon and the value "10", and "Número de años" with a calendar icon and the value "6". At the bottom of the input section are two buttons: "Calcular salario" with a calculator icon and "Limpiar" with an eraser icon.

Ejecución del Problema 2

4.3.3. Salida Esperada



Salida del Problema 2

4.3.4. Ejercicio 4.2: El náufrago satisfecho

Descripción:

“El náufrago satisfecho” ofrece hamburguesas sencillas (S) a \$20, dobles (D) a \$25 y triples (T) a \$28. La empresa acepta tarjetas de crédito con un cargo de 5 % sobre la compra. Suponiendo que los clientes adquieren N hamburguesas, realice el algoritmo para determinar cuánto deben pagar.

Pseudocódigo:

Inicio

Leer N // Cantidad de hamburguesas

Leer usaTarjeta // Verdadero o Falso

totalCosto = 0

Para i = 1 Hasta N Hacer

Leer tipoHamburguesa // Puede ser S, D, T

Si tipoHamburguesa == "S" Entonces

totalCosto = totalCosto + 20

Sino Si tipoHamburguesa == "D" Entonces

totalCosto = totalCosto + 25

Sino Si tipoHamburguesa == "T" Entonces

totalCosto = totalCosto + 28

FinSi

FinPara

cargoTarjeta = 0

Si usaTarjeta == Verdadero Entonces

cargoTarjeta = totalCosto * 0.05

FinSi

totalAPagar = totalCosto + cargoTarjeta

Imprimir "Subtotal hamburguesas: \$", totalCosto

Imprimir "Cargo por tarjeta: \$", cargoTarjeta

Imprimir "Total a Pagar: \$", totalAPagar

Fin

Diagrama Nassi-Shneiderman:

Leer N, usaTarjeta			
totalCosto ← 0			
Para i = 1 Hasta N			
Leer tipoHamburguesa			
tipoHamburguesa == 'S'			
Sí		No	
totalCosto ← totalCosto + 20		tipoHamburguesa == 'D'	
∅	Sí		No
	totalCosto ← totalCosto + 25		tipoHamburguesa == 'T'
	∅		No
		totalCosto ← totalCosto + 28	∅
cargoTarjeta ← 0			
usaTarjeta == Verdadero			
Sí		No	
cargoTarjeta ← totalCosto * 0.05		∅	
totalAPagar ← totalCosto + cargoTarjeta			
Imprimir totalAPagar			

Código:

```
lib/model/hamburguesas_model.dart

void calcularPago(List<String> hamburguesasCompradas, bool usaTarjeta) {
  double totalCosto = 0.0;
  for (String tipo in hamburguesasCompradas) {
    if (tipo == 'S') totalCosto += 20;
    else if (tipo == 'D') totalCosto += 25;
    else if (tipo == 'T') totalCosto += 28;
  }

  double cargoTarjeta = usaTarjeta ? (totalCosto * 0.05) : 0.0;
  double totalAPagar = totalCosto + cargoTarjeta;
}
```

Descripción: La función recibe una lista con los tipos de hamburguesas y un valor booleano para la tarjeta. Itera la lista sumando los costos según el tipo, y al final aplica un recargo del 5 % sobre el acumulado si es un pago con tarjeta.

4.3.5. Ejecución



Ejecución y salida del Problema 3

4.3.6. Salida Esperada

2:55

Hamburguesas

Menú: El Náufrago Satisfecho

Sencillas (\$20) 0 1

Dobles (\$25) 0 1

Triples (\$28) 0

¿Pagar con Tarjeta de Crédito? (+5%) ☒

Calcular Total

TICKET DE COMPRA

Sencillas (1): \$20.00
Dobles (1): \$25.00
Triples (0): \$0.00

Subtotal: \$45.00
Cargo Tarjeta (5%): \$2.25

TOTAL A PAGAR: \$47.25

Salida del Problema 3

4.3.7. Ejercicio 4.3: Contar cantidades

Descripción:

Se requiere un algoritmo para determinar, de N cantidades, cuántas son cero, cuántas son menores a cero, y cuántas son mayores a cero. Realice el algoritmo utilizando el ciclo apropiado.

Pseudocódigo:

Inicio

Leer N // Número total de cantidades

contadorCeros = 0

contadorMenoresCero = 0

contadorMayoresCero = 0

Para i = 1 Hasta N Hacer

Leer cantidad

Si cantidad == 0 Entonces

contadorCeros = contadorCeros + 1

Sino Si cantidad < 0 Entonces

contadorMenoresCero = contadorMenoresCero + 1

Sino

contadorMayoresCero = contadorMayoresCero + 1

FinSi

FinPara

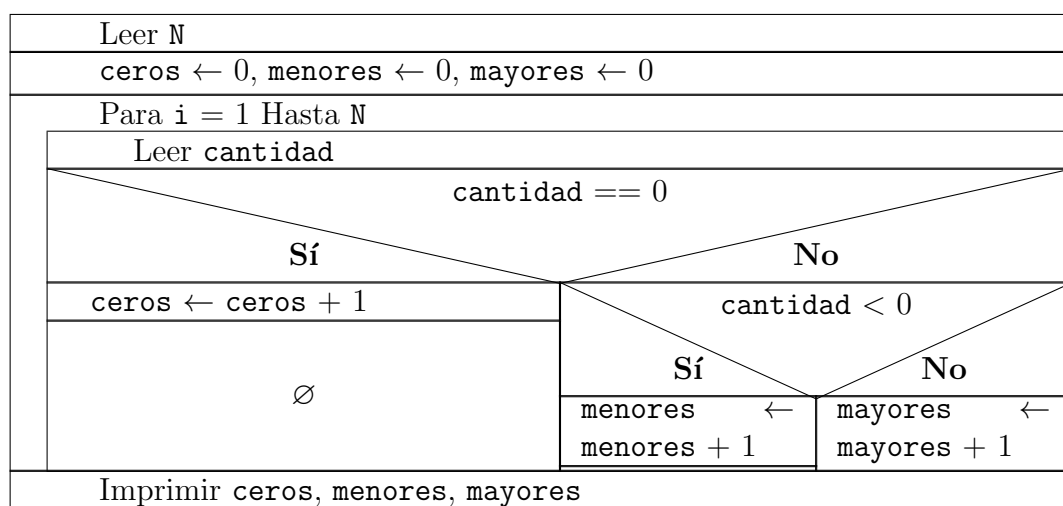
Imprimir "Cantidades iguales a cero: ", contadorCeros

Imprimir "Cantidades menores a cero: ", contadorMenoresCero

Imprimir "Cantidades mayores a cero: ", contadorMayoresCero

Fin

Diagrama Nassi-Shneiderman:



Código:

```
lib/model/cantidades_model.dart

void analizarCantidades(List<int> cantidades) {
  int ceros = 0, menores = 0, mayores = 0;

  for (int cantidad in cantidades) {
    if (cantidad == 0) ceros++;
    else if (cantidad < 0) menores++;
    else mayores++;
  }
  // Retorna o almacena los contadores
}
```

Descripción: Se recorre una lista de números evaluando cada elemento mediante condicionales `if-else` para determinar si es neutro (cero), negativo o positivo, incrementando el contador correspondiente en cada caso.

4.3.8. Ejecución



Ejecución y salida del Problema 4

4.3.9. Salida Esperada



Salida del Problema 4

4.3.10. Ejercicio 4.8: Promoción de artículos

Descripción:

Realice el algoritmo para determinar cuánto pagará una persona que adquiere N artículos, los cuales están de promoción: - Precio $\leq$ \$200: descuento de 15 % - \$100 < precio $\leq$ \$200: descuento de 12 % - Precio $\leq$ \$100: descuento de 10 % Muestre el costo y descuento de cada artículo, y cuánto pagará por el total.

Pseudocódigo:

Inicio

```
Leer N // Cantidad de artículos
totalPagoGlobal = 0
```

```
Para i = 1 Hasta N Hacer
```

```
    Leer precioArticulo
    descuentoArticulo = 0
```

```
    Si precioArticulo >= 200 Entonces
        descuentoArticulo = precioArticulo * 0.15
    Sino Si precioArticulo > 100 Entonces
        descuentoArticulo = precioArticulo * 0.12
    Sino
        descuentoArticulo = precioArticulo * 0.10
    FinSi
```

```
    precioFinalArticulo = precioArticulo - descuentoArticulo
    totalPagoGlobal = totalPagoGlobal + precioFinalArticulo
```

```
    Imprimir "Artículo ", i
    Imprimir " Precio original: $", precioArticulo
    Imprimir " Descuento: $", descuentoArticulo
    Imprimir " Precio final: $", precioFinalArticulo
FinPara
```

```
Imprimir "Total a pagar por todos los artículos: $", totalPagoGlobal
Fin
```

Diagrama Nassi-Shneiderman:

Leer N		
totalPagoGlobal $\leftarrow$ 0		
Para i = 1 Hasta N		
Leer precioArticulo		
descuentoArticulo $\leftarrow$ 0		
precioArticulo $\geq$ 200		
Sí		No
descuentoArticulo $\leftarrow$ precioArticulo * 0.15	precioArticulo > 100	
$\emptyset$	Sí	No
	descuentoArticulo $\leftarrow$ precioArticulo * 0.12	descuentoArticulo $\leftarrow$ precioArticulo * 0.10
precioFinalArticulo $\leftarrow$ precioArticulo - descuentoArticulo		
totalPagoGlobal $\leftarrow$ totalPagoGlobal + precioFinalArticulo		
Imprimir totalPagoGlobal		

Código:

```
lib/model/promocion_model.dart

void procesarArticulos(List<double> precios) {
  double totalGlobal = 0.0;

  for (double precio in precios) {
    double descuento = 0.0;
    if (precio >= 200) descuento = precio * 0.15;
    else if (precio > 100) descuento = precio * 0.12;
    else descuento = precio * 0.10;

    double precioFinal = precio - descuento;
    totalGlobal += precioFinal;
  }
  // 'totalGlobal' contiene lo que se pagará en total
}
```

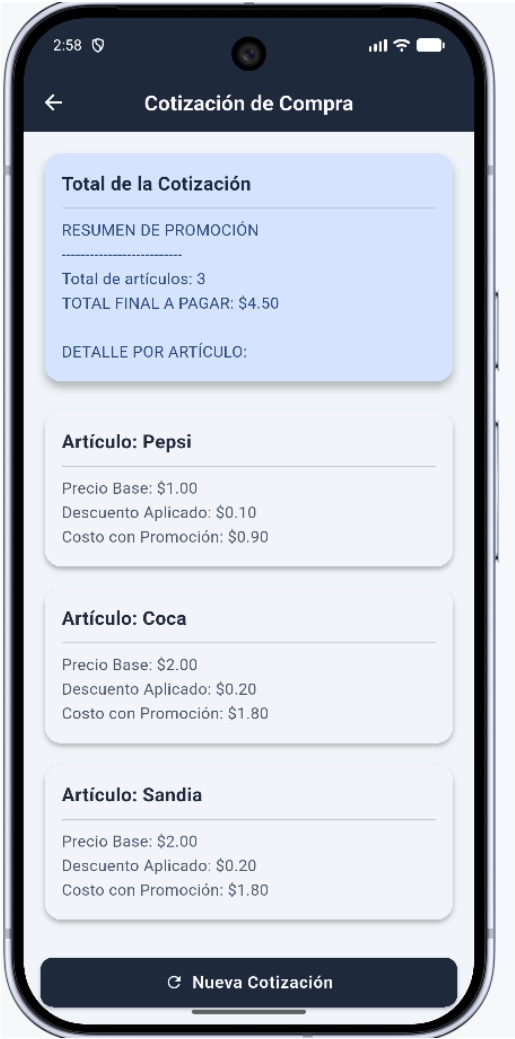
Descripción: La lógica analiza el precio individual de cada artículo iterado para aplicarle su descuento respectivo. Resta el descuento al costo base y va acumulando los precios resultantes en una variable global que dictará el total a pagar.

4.3.11. Ejecución



Ejecución y salida del Problema 5

4.3.12. Salida Esperada



Salida del Problema 5

5. Conclusiones

- La implementación de un *Splash screen* mejora significativamente la experiencia del usuario, ocultando tiempos de carga iniciales bajo una estética de marca que aplica un componente UI fundamental en el desarrollo móvil.
- Las instrucciones de navegación de Flutter, como `push`, `pop` y `pushNamed`, proveen un control exhaustivo sobre la pila de interfaces, logrando transiciones asíncronas fiables entre los ejercicios propuestos.
- Se completó con éxito el análisis secuencial y lógico de todos los problemas matemáticos y financieros a través del diseño de pseudocódigos altamente estructurados, validando su comportamiento antes de escribir instrucciones propias de un marco de trabajo.

6. Referencias

- Google (2024). *Flutter Documentation*. Recuperado de <https://docs.flutter.dev/>
- Google (2024). *Dart Language Overview*. Recuperado de <https://dart.dev/overview>
- Google (2024). *Android Studio User Guide*. Recuperado de <https://developer.android.com/studio/intro>

7. Anexos

7.1. Repositorio de código fuente

7.1.1. Enlace principal

- https://github.com/LeonelTQL/moviles/tree/main/pry_lab2